LLM-Elicited Preferences in an EDSL School-Choice Machine

School-choice deferred-acceptance pilot

October 11, 2026

Abstract

This pilot connects LLM-elicited student preferences to a declarative EDSL implementation of many-to-one deferred acceptance. Twelve student personas rank four schools with three seats each. The same elicited preference profile and the same seeded school priorities are used in student-proposing and school-proposing runs. Both outcomes are complete, capacity-feasible, and stable. Student-proposing DA gives all twelve personas their first choice; school-proposing DA gives seven their first choice and has mean student rank 1.75. This is a small mechanism demonstration: it tests the LLM-to-Machine data path and the proposer-side distinction, not the validity of LLM-generated preferences as a model of real families.

1 Question and design

Can EDSL's shared-state Machine represent the state transitions of school-choice deferred acceptance while receiving preferences elicited from LLM interviews? This pilot uses a deliberately small balanced market: twelve students, four schools, and capacity three at every school. Each student receives a short persona describing location and educational interests. Each school has a short catalog description. One LLM ranking response supplies each student's complete strict ranking of the four schools.

The preference survey is run locally from the project process through the configured OpenAI provider using EDSL's `disable_remote_inference=True`. The model is `gpt-4o-mini`, temperature 0.2. "Locally" here describes where EDSL executes the interview orchestration and handles credentials; the model inference itself is served by the configured provider. No local inference server is involved.

School priorities are strict random orderings generated from seed 42017. The two mechanism runs consume the exact same LLM preference profile and priority lists. Only the proposer side changes. The complete inputs, rankings, proposal logs, outcomes, and summary metrics are saved in `preferences.json` and `results.json` beside this report.

2 Formal market and stability

Let S be the finite student set and C the school set. School c has capacity $q_c = 3$. Each student s has a strict preference order $\succ_s$ over all schools, and each school c has a strict priority order $\succ_c$ over all students. A matching μ assigns each student to one school and no school more than its capacity.

A pair (s, c) blocks μ when student s prefers c to their assignment and either c has an unfilled seat or c prioritizes s over at least one currently assigned student:

$$c \succ_s \mu(s) \quad \text{and} \quad \big(|\mu(c)| < q_c \ \text{ or } \ \exists s' \in \mu(c) : s \succ_c s'\big).$$

A matching is stable if it has no blocking pair.

In student-proposing DA, each unheld student proposes to their most-preferred school not yet approached. Each school holds its highest-priority applicants up to capacity and rejects the remainder. In school-proposing DA, each school with an open seat proposes to its highest-priority student not yet

approached. Each student holds their favorite offer to date; a rejected school advances down its priority list. Strict preferences, complete lists, finite participants, and no repeated proposals imply termination.

3 Machine state and embedded loop

The EDSL Machine stores the market instance as constants: participants, capacities, preferences, school priorities, and rank maps. Its single mutable field, `allocation`, is a typed record containing the proposal queue, student assignments, tentative school holds, both proposer cursor maps, the proposal log, and a completion flag. The full procedure is an EDSL `iterate` expression in `close_effects`. In each iteration, a pure expression reads the current allocation, selects the next eligible proposer, forms the proposal, returns a complete next allocation record, and records the outcome. The bounded loop terminates when its declared queue or active-school condition is true. No named Python callback is used (`algorithms=()`).

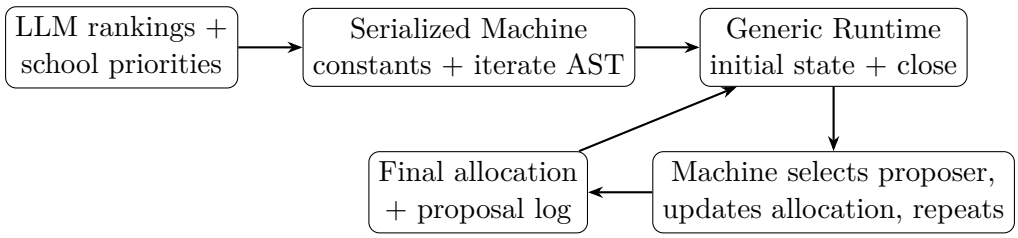


Figure 1: The matching loop is in the serialized Machine; Python invokes only the generic Runtime lifecycle.

The student-proposing loop removes the first student from its queue, selects that student's next ranked school, sorts the school's incumbents plus new applicant by priority, retains up to capacity, requeues rejected students, and advances the student's cursor. The school-proposing loop selects the first school with an open seat and an untried student, makes that offer, lets the student keep their preferred school, and advances the school's cursor. Each loop has a finite explicit bound. The close transition computes the final allocation atomically; intermediate states are preserved as proposal-log entries rather than committed one by one.

4 Run procedure

First, the survey defines one `QuestionRank` with all four school IDs as options and requires a complete ranking. Second, twelve persona agents answer with their individual profile inserted into the prompt. Third, each response is validated as a permutation of all four schools and saved. Finally, each proposer-side Machine embeds that saved preference profile and the seeded school priorities. The generic runner initializes the Runtime and invokes the standard close operation; it has no school-choice-specific proposer loop.

The core local-execution setting is:

```
results = (survey.by(agents)
    .by(Model(model_name, service_name="openai", temperature=0.2))
    .run(cache=False, disable_remote_cache=True,
        disable_remote_inference=True, max_concurrency=4,
        stop_on_exceptions=True))
```

The same saved preferences are passed to both Machine definitions:

```
machine = build_machine(seed, proposer_side, preferences)
runtime = Runtime()
state = runtime.initial_state(machine)
final = runtime.close_result(machine, state)
```

5 Results

Outcome	Student-proposing	School-proposing
First-choice assignments	12/12	7/12
Mean student rank (1 = best)	1.00	1.75
Proposal transitions	12	22
Blocking pairs	0	0
Seats filled	12/12	12/12

Student-proposing DA produces the student-optimal stable matching for this fixed market, so it is weakly better for every student than any other stable matching, including the school-proposing outcome. In this particular preference profile, all students receive their first-ranked school in the student-proposing outcome. In school-proposing DA, five students receive a lower-ranked school: four are assigned their third choice and one their second choice. Both outcomes satisfy capacities and have no blocking pairs.

The unusually clean student-side outcome is a property of this specific elicited ranking profile together with the priority draws: every student receives the school at the top of their list, and each school receives exactly three first-choice applicants. It should not be read as a general property of school choice or of this prompt. Repeated LLM interviews, alternative seeds, and human preference data would be needed to characterize the distribution of outcomes.

The Machine logged 12 student proposals and 22 school proposals. This difference reflects distinct proposal protocols and does not by itself indicate computational performance. The same saved rankings are embedded in both serialized Machines; each Machine includes its proposer selection and repeat-until loop in its close effect.

6 Interpretation and limitations

This pilot demonstrates an end-to-end integration: a structured EDSL ranking question produces machine-readable strict preferences, the preferences are validated and persisted, two serialized Machine configurations run against the same market, and independently checked outcomes are stable. It also makes proposer-side consequences visible in a compact, auditable example.

It does not establish that LLM personas behave like real students or families. Persona descriptions are authored inputs, not sampled people; a single completion per persona cannot measure response variability; the model's internal assumptions may affect rankings; and the priorities are seeded synthetic rankings rather than an institutional admissions policy. The result is a software and mechanism pilot, not an empirical policy evaluation. A stronger next study would collect multiple independent ranking draws per persona, vary model and prompt, evaluate sensitivity to persona and school-description wording, and report outcome distributions across many priority seeds.

A Reproduction and artifacts

From the repository root, run the LLM pilot with:

```
PYTHONPATH=edsl_replication/studies/school_choice_deferred_acceptance \\
  python3 edsl_replication/studies/school_choice_deferred_acceptance/llm_pilot.py
```

The command requires the provider key to be configured in the local environment. It writes preference and outcome JSON to `pilot_llm_01/`. To rerun only the deterministic matching step on the saved profile, pass `-reuse-preferences`. The EDSL CAS object store is directed into `pilot_llm_01/edsl_objects/` so the runner's local result persistence remains inside the study artifact.

- `machine_definition.py`: Machine constants, typed allocation, and bounded EDSL loops.
- `run_pilot.py`: generic Runtime initialization and close, followed by independent capacity/stability checks.
- `llm_pilot.py`: persona survey, ranking validation, local EDSL run, and LLM-profile comparison.
- `preferences.json`: the 12 elicited strict rankings.
- `results.json`: both Machine outcomes, priorities, event logs, and metrics.

See the companion `machine_technical_note.pdf` for syntax-highlighted source excerpts and the complete details of the serialized loop, state record, step bounds, and generic Runtime call.